

Week 2

Revision

Guide

Feature Work & Core ML Foundations · Days 08 – 15

Concepts · Code · Real World Problems · Interview Prep

WHAT'S INSIDE

DAY 08 Week 1 Revision Recap	DAY 09 Feature Engineering
DAY 10 Feature Selection	DAY 11 Scaling & Encoding
DAY 12 How ML Actually Works	DAY 13 Linear Regression
DAY 14 Logistic Regression	DAY 15 Decision Trees
DAY BONUS MNC Interview Q&A	

Before We Begin

Week 1 built the foundation: understanding your data before touching a model. Here is the one-line summary of each day, so Week 2 builds on solid ground.

Day	Topic	One-Line Takeaway
01	What is Data	Structured, semi-structured, unstructured. Each needs its own pipeline.
02	Data Types	Numerical, ordinal, categorical, boolean. Wrong dtype = wrong model assumptions.
03	Descriptive Stats	Mean, median, std, skew, kurtosis. $ \text{skew} > 1$ means log-transform it.
04	EDA	Shape -> univariate -> bivariate -> target balance -> pairplot. Always in this order.
05	Missing Data	MCAR, MAR, MNAR. Fit imputer on train only. Add missingness indicators.
06	Outliers	IQR, Z-score, Isolation Forest. Context decides remove vs keep, not the algorithm.
07	Data Cleaning	Duplicates, formatting, dtypes, contradictions, structural errors. Validate with assertions.

WEEK 2 ROADMAP

Week 1 was about understanding data. Week 2 is about preparing it for models and building your first real algorithms: Feature Engineering, Feature Selection, Scaling and Encoding, then the core ML loop itself -- How ML Actually Works, Linear Regression, Logistic Regression, and Decision Trees.

SELF-CHECK BEFORE STARTING WEEK 2

Without notes: explain the difference between MCAR, MAR, and MNAR with one example each. Name 3 outlier detection methods. List the 5 data cleaning issues. If any feel shaky, revisit that day before continuing.

Creating Signal From Raw Columns

THE CORE IDEA

Feature engineering means creating NEW columns from existing ones that make patterns easier for a model to find. The raw data already contains the signal — engineered features just make it visible.

Datetime Decomposition Extract hour, day_of_week, is_weekend from a timestamp A single "timestamp" column hides multiple signals <i>Used by Uber for surge pricing</i>	Binning Convert continuous age into buckets: 18-25, 26-35, 36-50, 50+ Reduces noise, helps tree models split cleanly <i>Common in credit scoring</i>	Interaction Terms Multiply or combine two features: $\text{price_per_sqft} = \text{price} / \text{sqft}$ Captures relationships a single column cannot <i>Real estate, pricing models</i>
--	--	--

REAL WORLD — UBER SURGE PRICING

UBER ENGINEERING

Uber extracts "hour of day" and "is_weekend" from a raw pickup timestamp. These two engineered features feed directly into the surge pricing model. The raw timestamp alone is nearly useless to a model — it has no concept of "rush hour" or "Friday night." The engineered features encode exactly that.

KEY SYNTAX

```
# Datetime decomposition
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)

# Binning a continuous feature
df['age_bucket'] = pd.cut(df['age'],
    bins=[0, 25, 35, 50, 100],
    labels=['18-25', '26-35', '36-50', '50+'])

# Interaction term
df['price_per_sqft'] = df['price'] / df['sqft']
```

THE PRODUCTION CHECK

Always measure model accuracy BEFORE and AFTER adding an engineered feature. "It feels like it should help" is not evidence. In the Day 9 notebook, adding hour_of_day + is_weekend to a tip-prediction model improved R² from 0.41 to 0.58 — a real, measured gain, not a guess.

GOLDEN RULE

A raw timestamp tells a model nothing about "Friday night surge." Decomposing it into hour, day_of_week, and is_weekend turns one opaque column into multiple features the model can actually use to find patterns.

INTERVIEW QUESTIONS

DAY 09 · Q1

UBER — ML ENGINEER ROUND

You have a raw "order_timestamp" column. How would you turn it into useful features for a demand forecasting model?

ANSWER DIRECTION

Decompose into hour_of_day, day_of_week, is_weekend, and is_holiday. Each captures a different demand pattern — hour_of_day captures daily rhythm, day_of_week captures weekly patterns, is_weekend and is_holiday capture special-case spikes. For longer-term models, also extract month or day_of_month to capture seasonal trends. Always validate by checking if model performance actually improves after adding these — measure before and after.

DAY 09 · Q2

ZOMATO — DATA SCIENTIST ROUND

When does feature engineering risk introducing data leakage? Give an example.

ANSWER DIRECTION

Leakage happens when an engineered feature uses information that would not be available at prediction time. Example: creating "average_delivery_time_for_this_restaurant" using data that includes the order you are trying to predict. Fix: when computing rolling averages or aggregates, only use data from BEFORE the prediction timestamp (a backward-looking window). Always ask: "would I have known this value at the moment I needed to make this prediction?"

Fewer, Better Features

THE CORE IDEA

More features is not always better. Irrelevant or redundant features add noise, slow training, and in linear models, distort coefficients. Feature selection removes features that do not help — keeping only what carries real signal.

Filter Methods	Wrapper Methods	Embedded Methods
Correlation with target Chi-square test for categorical features Fast, model-independent <i>Run first, always</i>	Recursive Feature Elimination (RFE) Trains model repeatedly, drops weakest feature each round <i>Slower, more accurate</i>	Lasso (L1) regression shrinks weak coefficients to exactly 0 Feature importance from tree models <i>Built into the model itself</i>

REAL WORLD — META AD CLICK MODEL

META ENGINEERING

Meta removed low-variance and highly-correlated features from their ad click prediction model. Result: training time dropped by 40% with no meaningful loss in accuracy. The removed features were not wrong — they were redundant. Another feature already carried the same signal.

KEY SYNTAX

```
# Filter: correlation with target
corr_with_target = df.corr()['target'].abs().sort_values(ascending=False)

# Wrapper: RFE
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
selector = RFE(LogisticRegression(max_iter=1000), n_features_to_select=5)
selector.fit(X_train, y_train)
selected = X.columns[selector.support_]

# Embedded: Lasso
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1).fit(X_train, y_train)
# coefficients near 0 -> feature contributes little
```

THE PRODUCTION CHECK

Always compare all 3 methods on the SAME dataset and note which features each one keeps. They often disagree -- a feature Lasso drops might be one RFE keeps. There is no single "correct" answer; the right method depends on the model you are deploying.

GOLDEN RULE

A feature with near-zero variance (the same value for almost every row) carries almost no information, regardless of how "important" it sounds conceptually. Check variance and correlation BEFORE running any wrapper or embedded method -- it is the cheapest first filter.

INTERVIEW QUESTIONS

DAY 10 · Q1

META — ML ENGINEER ROUND

You have 200 features and a model that takes 6 hours to train. How would you reduce the feature set without hurting accuracy much?

ANSWER DIRECTION

Start with cheap filters: drop near-zero-variance features and features with correlation above 0.95 to another feature (multicollinearity). Then use an embedded method like Lasso or a tree-based feature importance ranking to find a smaller subset (e.g., top 30-50 features). Validate by comparing model accuracy on the full feature set vs the reduced set on a held-out test set -- if accuracy drops by less than 1-2% but training time drops by 40%+, the tradeoff is usually worth it.

DAY 10 · Q2

SWIGGY — DATA SCIENTIST ROUND

RFE and Lasso disagree on which features matter. How do you decide which to trust?

ANSWER DIRECTION

Neither is universally "correct" -- they optimize for different things. RFE wraps around a specific model and measures its performance directly, so it reflects that model's behavior. Lasso uses L1 regularization and tends to arbitrarily pick one feature among a correlated group, dropping the others to zero. If two features are highly correlated, Lasso might keep one and drop the other, while RFE might keep both because removing either barely changes performance. The right answer: match the selection method to the model you will actually deploy, and always validate with held-out test accuracy, not just the selection method's internal ranking.

The 2 Steps That Silently Break Models

THE CORE IDEA

KNN and SVM compute distance between points. Without scaling, a feature with range 130,000 (income) completely dominates a feature with range 38 (age). The model effectively ignores age -- not because it is unimportant, but because its scale is invisible next to income.

	MinMaxScaler	StandardScaler
Output	0 to 1	mean=0, std=1
Formula	$(x - \min) / (\max - \min)$	$(x - \text{mean}) / \text{std}$
Outliers	Breaks badly	More robust
Use when	Neural nets, bounded inputs	Most ML algorithms (default)

REAL WORLD — THE PROOF

FROM THE DAY 11 NOTEBOOK

Same KNN model, same data, same k=5. Accuracy WITHOUT scaling: 77.5%. Accuracy WITH StandardScaler: 100%. The only difference was one line of code -- StandardScaler().fit_transform(). That is the entire impact of ignoring scale.

3 ENCODING STRATEGIES

Encoding	Use When	Risk If Misused
Label Encoding	Target variable (y) only	Implies Mumbai(2) > Delhi(1) > Bangalore(0) -- false
One-Hot Encoding	Nominal features: city, brand, job type	Too many columns on high-cardinality features
Ordinal Encoding	Ordered features: small < medium < large	Wrong order = model learns the OPPOSITE relationship

KEY SYNTAX

```
# Always fit on TRAIN only, transform test separately
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), ['age', 'income']),
    ('nom', OneHotEncoder(drop='first'), ['city']),
    ('ord', OrdinalEncoder(categories=[['low', 'med', 'high']]), ['risk']),
])

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', KNeighborsClassifier(n_neighbors=5))
])
pipeline.fit(X_train, y_train) # scaling + encoding + model, ONE call
```

THE PRODUCTION TRAP — DEPLOYMENT SCALER BUG

A data scientist fits StandardScaler on training data and exports the model with joblib. The deployment engineer loads the model and feeds RAW (unscaled) data into it. No error is thrown. Predictions are silently wrong for 2 weeks until business analysts notice. Fix: always use a Pipeline -- scaler and model saved as ONE object. The preprocessing runs automatically on every prediction; you cannot forget to scale.

INTERVIEW QUESTIONS

DAY 11 · Q1

GENERAL MLE ROUND

sklearn's KNN gives different accuracy with and without StandardScaler. Why? When would you NOT need to scale?

ANSWER DIRECTION

KNN computes Euclidean distance. Without scaling, features with larger numeric ranges dominate the distance metric -- age (range 38) becomes invisible next to income (range 130,000). Scaling puts every feature on the same footing. You would NOT need scaling for tree-based models (Decision Tree, Random Forest, XGBoost, LightGBM) -- they split on thresholds, not distances, so the absolute value of a feature does not affect where a split is placed. Critical detail interviewers look for: fit the scaler on training data only, then transform the test set separately -- fitting on the full dataset is data leakage.

DAY 11 · Q2

FLIPKART — ML ENGINEER ROUND

What's the difference between MinMax and Standard scaling? When does MinMax fail?

ANSWER DIRECTION

MinMax maps values to a $[0,1]$ range using $(x-\min)/(\max-\min)$. StandardScaler centers data to $\text{mean}=0$, $\text{std}=1$ using $(x-\text{mean})/\text{std}$. MinMax fails when outliers are present: one extreme value shifts the min or max, compressing all other values into a tiny range -- e.g., 5 salaries of 30k-90k get compressed to 0.005-0.017 once a 5,000,000 outlier is added. StandardScaler is more robust because mean and std are less distorted by a single extreme value. Default to StandardScaler; use MinMax only for neural network inputs or naturally bounded features with no outliers.

The Learn-From-Data Loop

THE CORE IDEA

A model is a function with adjustable numbers: $y_{\text{hat}} = w \cdot x + b$. "Training" means finding the values of w and b that make predictions match reality. Every model -- from linear regression to a 70-billion-parameter LLM -- follows this exact loop.

THE 4-STEP LOOP

Step	What Happens
1. Predict	Make predictions using current w, b
2. Measure	Compute the LOSS -- how wrong the predictions are (MSE)
3. Gradient	Compute the slope of the loss surface -- which direction makes loss WORSE
4. Step	Move w and b in the OPPOSITE direction (downhill). Repeat thousands of times.

REAL WORLD — FROM THE DAY 12 NOTEBOOK

PROOF: HAND-BUILT GD MATCHES SKLEARN

Gradient descent written from scratch in plain numpy, trained for 100 epochs on house price data: $w=46.190367$, $b=229.196420$. sklearn `LinearRegression.fit()` on the SAME data: $w=46.190367$, $b=229.196420$. Identical to 6 decimal places. sklearn `.fit()` runs this exact loop, written in optimized C.

THE LEARNING RATE CONTROLS EVERYTHING

Learning Rate	Loss Start -> End (50 epochs)	Result
0.001 (too small)	55,239 -> 45,501	Training crawls
0.1 (just right)	55,239 -> 575	Smooth convergence
1.5 (too large)	55,239 -> infinity	DIVERGES -- loss explodes

KEY SYNTAX

```
# Gradient descent from scratch
def gradient_descent(X, y, learning_rate=0.1, epochs=100):
    w, b = 0.0, 0.0
    m = len(X)
    for epoch in range(epochs):
        y_hat = w * X + b
        loss = np.mean((y_hat - y) ** 2)      # MSE
        dw = (2/m) * np.sum((y_hat - y) * X)  # gradient
        db = (2/m) * np.sum(y_hat - y)
        w -= learning_rate * dw                # step downhill
        b -= learning_rate * db
    return w, b
```

THE PRODUCTION TRAP — NAN LOSS

An MLE trains a neural network. Loss goes 4.2, 3.9, then nan by epoch 3. The team spends a day checking for bad data and corrupted labels -- the data is fine. The learning rate was too high: the first steps overshoot the minimum so badly that weights overflowed to infinity, then nan. Fix: lower the learning rate by 10x. This single hyperparameter causes more "model is not learning" debugging sessions than almost anything else.

INTERVIEW QUESTIONS

DAY 12 · Q1

GENERAL MLE ROUND

Your model's loss becomes nan during training. What do you check first?

ANSWER DIRECTION

Learning rate. Lower it by 10x before checking data, architecture, or labels. A nan loss almost always means the gradient steps overshoot the minimum so severely that weights grew unboundedly, overflowed to infinity, and then became nan once arithmetic on infinity occurred ($\text{inf} - \text{inf} = \text{nan}$). This is reproducible: train the same model with $\text{lr}=1.5$ and watch loss go from 55,239 to infinity within 7 epochs, then train with $\text{lr}=0.1$ and watch it converge smoothly to 575. Same data, same model -- only the learning rate changed.

DAY 12 · Q2

GENERAL MLE ROUND

Explain what sklearn's .fit() actually does, in your own words, without saying "it trains the model."

ANSWER DIRECTION

.fit() runs gradient descent (or a closed-form solution, for simple linear regression). It starts with initial parameter values (often 0), repeatedly computes predictions, measures the loss (MSE for regression), computes the gradient of that loss with respect to each parameter, and updates each parameter by stepping in the direction that reduces loss. This repeats until the loss stops improving meaningfully (convergence) or a maximum iteration count is reached. The "learned" w and b values stored in `model.coef_` and `model.intercept_` are the result of this iterative search -- not looked up from a table, but found by following the slope of the loss surface downhill.

Reading Coefficients Correctly

THE CORE IDEA

With multiple features, each gets its own weight: $\hat{y} = w_1 \text{sqft} + w_2 \text{bedrooms} + w_3 \text{age} + b$. The model learns all weights at once. Reading them correctly -- and knowing when NOT to trust them -- is the actual skill.

R², RMSE, MAE — WHAT THEY ACTUALLY MEAN

Metric	Plain-English Meaning
R² = 0.769	Model explains 77% of price variation. Remaining 23% is noise or missing features.
RMSE = 21.5 lakhs	Typical error size, same units as price. Punishes large errors more (squared).
MAE = 16.6 lakhs	Average absolute error. Less sensitive to a few large outlier errors than RMSE.
RMSE >> MAE	A signal that a few predictions are off by a LOT, pulling RMSE up.

REAL WORLD — THE MULTICOLLINEARITY TRAP

FROM THE DAY 13 NOTEBOOK

Added a near-duplicate of "sqft" (correlation 0.9999) to the model. Across 5 different train/test splits, the sqft coefficient swung from -0.15 to +0.07 -- even flipping sign. But the SUM of both coefficients stayed locked at 0.121 every time. Predictions and R² stayed unchanged. Only individual coefficients became meaningless for interpretation.

KEY SYNTAX

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)

r2 = r2_score(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
mae = mean_absolute_error(y_test, y_pred)

# ALWAYS check correlation before trusting coefficients
corr_matrix = X.corr()
# flag any pair with |correlation| > 0.8
```

PRODUCTION RULE

Linear regression coefficients are only meaningful as "effect size" when features are not strongly correlated with each other. For PURE PREDICTION, multicollinearity often does not hurt accuracy at all. For INTERPRETATION or business explanation, check the correlation matrix first -- correlation above 0.8-0.9 between two features is a red flag.

INTERVIEW QUESTIONS

DAY 13 · Q1

GENERAL MLE ROUND

Two features in your model are 95% correlated. What happens to your coefficients, and does it matter?

ANSWER DIRECTION

Coefficients become unstable -- they can flip sign across retrains, sometimes reversing direction entirely. Predictions and R^2 stay nearly unchanged because the model can still use the combined signal from both features together. It matters ONLY if you are using coefficients for interpretation or business explanation ("Source A square footage adds value, Source B subtracts value" -- a conclusion that could flip with a different random seed). For pure prediction, leave both features in. For interpretation, drop one or combine them with PCA first.

DAY 13 · Q2

ZILLOW-STYLE — DATA SCIENCE ROUND

Your model has RMSE=21.5 but MAE=16.6. What does the gap between these two numbers tell you?

ANSWER DIRECTION

RMSE squares errors before averaging, so large individual errors are punished disproportionately. MAE averages absolute errors without squaring. If RMSE is noticeably higher than MAE, it means a SUBSET of predictions have unusually large errors -- the model is mostly accurate but occasionally very wrong on specific cases. Action: look at the residual plot and identify which rows have the largest errors. These often correspond to edge cases (very large houses, unusual locations) that may need separate handling or additional features.

Probabilities, Not Numbers

THE CORE IDEA

Linear regression can output ANY number -- including -0.22 or 1.23, which mean nothing as probabilities. The sigmoid function squashes $w \cdot x + b$ into the range (0,1), no matter how extreme the input. $\text{sigmoid}(z) = 1 / (1 + e^{-z})$.

WHY LINEAR REGRESSION FAILS HERE

FROM THE DAY 14 NOTEBOOK

Fit linear regression on a 0/1 target. 19 of 100 predictions came out below 0; 15 came out above 1. Adding ONE outlier ($x=50$) shifted the linear model's predictions for $x=2, 5, 8$ toward 0.3-0.6 across the board -- dragged off course by a single point. The logistic model stayed sensible: 0.0008, 0.53, 0.999 for the same three points.

THE SIGMOID FUNCTION

$z = w \cdot x + b$	$\text{sigmoid}(z)$
-10.0	0.0000
-1.0	0.2689
0.0	0.5000 <- decision boundary
1.0	0.7311
10.0	1.0000

THE ACCURACY PARADOX

FROM THE DAY 14 NOTEBOOK — FRAUD DETECTION

1000 transactions, 5.1% fraud rate. Logistic regression model: 94.5% accuracy. Confusion matrix: caught 0 out of 10 actual fraud cases. A model predicting "not fraud" for EVERYTHING also scores 95% accuracy -- BETTER than the trained model. Lowering the threshold from 0.5 to 0.05: fraud caught goes from 0/10 to 5/10, but overall accuracy drops to 76.5%. The model did not change -- only the cutoff for calling something "fraud" changed.

KEY SYNTAX

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, roc_auc_score

model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
y_proba = model.predict_proba(X_test)[: , 1] # probability, not class

# Default threshold is 0.5 -- but it is a CHOICE, not a law
for threshold in [0.5, 0.3, 0.1, 0.05]:
    preds = (y_proba >= threshold).astype(int)
    print(confusion_matrix(y_test, preds))
```

INTERVIEW QUESTIONS

DAY 14 · Q1

GENERAL MLE ROUND

Your fraud model has 95% accuracy. Your manager is happy. What do you check before celebrating?

ANSWER DIRECTION

Check the class distribution first. If fraud is 5% of transactions, a model predicting "not fraud" for everything also hits 95% accuracy -- the model could be doing nothing useful. Look at the confusion matrix, precision, and recall for the fraud class specifically. If recall on fraud is near 0, the model is useless regardless of overall accuracy. Then discuss the threshold -- the default 0.5 is not a law, and for fraud, missing a fraud case (false negative) usually costs far more than a false alarm (false positive), so a lower threshold may be the correct business choice even if it lowers overall accuracy.

DAY 14 · Q2

GENERAL MLE ROUND

Why does logistic regression use sigmoid instead of just clipping linear regression output to [0,1]?

ANSWER DIRECTION

Clipping creates a flat, uninformative region: every input that would map to a negative or >1 value gets squashed to exactly 0 or exactly 1, losing all relative information between them (an input of -50 and -5 both become 0, even though they represent very different confidence levels). Sigmoid smoothly compresses the entire range of inputs into (0,1) with no flat zones -- every value of z maps to a distinct probability, and the gradient is well-defined everywhere, which is essential for gradient descent to work. Sigmoid is also less sensitive to outliers in the input, as shown when one extreme x value barely changed the logistic model's predictions but visibly shifted the linear model's.

Splits, Purity, and Depth

THE CORE IDEA

A tree repeatedly asks: "Which question splits this group into the PUREST possible subgroups?" Gini Impurity = $1 - p^2 - (1-p)^2$, where p is the fraction belonging to class 1. At $p=0.5$ (50/50 mix), impurity is at its maximum. At $p=0$ or $p=1$ (pure group), impurity is 0.

HOW A SPLIT IS CHOSEN — BY HAND

FROM THE DAY 15 NOTEBOOK — 8 LOAN APPLICANTS

Parent gini = 0.4688 (5 of 8 approved). Tried splitting credit_score at 4 thresholds. Split at ≤ 680 gave the lowest weighted gini (0.1875) -- left group [0,0,1,0] gini=0.375, right group [1,1,1,1] gini=0.0 (perfectly pure). sklearn, searching more candidate points on the same data, picked 690 -- same region, same answer.

THE DEPTH EXPERIMENT — OVERFITTING IN REAL NUMBERS

max_depth	Train Accuracy	Test Accuracy	Gap
1	74.4%	78.8%	-4.4 pts
3	83.4%	85.0%	-1.6 pts
5	90.3%	71.3%	+19.1 pts
10 / None	100.0%	63.7%	+36.3 pts

Run on 400 loan applications with 16% label noise (real data always has exceptions). At depth=10, the tree memorized the noise. 100% training accuracy here is a WARNING SIGN, not a success.

KEY SYNTAX

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree

tree = DecisionTreeClassifier(max_depth=3, criterion='gini', random_state=42)
tree.fit(X_train, y_train)

# Human-readable decision path -- the audit trail
print(export_text(tree, feature_names=feature_names))

# Feature importances
for name, imp in zip(feature_names, tree.feature_importances_):
    print(name, imp)
```

THE PRODUCTION TRAP — INSURANCE CLAIMS MODEL

An insurance company ships a claims model with no `max_depth` set (sklearn's default). Training accuracy hits 100%. Three months later, real-world accuracy is far lower, and compliance asks why a specific claim was denied. The answer is a 47-level-deep tree with thousands of leaves -- unreadable. Fix: constrain `max_depth` to 3-6. A shallow tree trades a little raw accuracy for a model that generalizes AND produces a decision path a human can read: "denied because `credit_score<=647.5 AND income>77,826.5 AND credit_score<=523.`"

INTERVIEW QUESTIONS

DAY 15 · Q1

GENERAL MLE ROUND

Your decision tree has 100% training accuracy. Is that good?

ANSWER DIRECTION

No -- it is a red flag. 100% training accuracy with no `max_depth` means the tree grew until every leaf was pure, which usually means it memorized noise and outliers along with real patterns. Check the test accuracy gap: if train is 100% and test is significantly lower, that is overfitting. Constraining `max_depth` (commonly 3-6) often IMPROVES test accuracy even though it lowers training accuracy. In regulated domains like insurance or credit scoring, a shallow tree is also REQUIRED for explainability -- a compliance officer needs to read the decision path.

DAY 15 · Q2

GENERAL MLE ROUND

Why does sklearn default to Gini impurity instead of entropy? Does it matter which you pick?

ANSWER DIRECTION

Gini is computationally cheaper -- it avoids logarithms, which entropy requires. In practice, the two measures rarely lead to meaningfully different trees; they are both maximized at $p=0.5$ and minimized at $p=0$ or $p=1$, just on slightly different numeric scales. For most production use cases, the choice between gini and entropy makes little practical difference -- `max_depth`, `min_samples_leaf`, and feature engineering have far more impact on the final tree than the impurity criterion.

Uber · Meta · Swiggy · Zomato · Flipkart · General MLE Rounds

Day · Company	Question	Key Points
09 · Uber	How would you turn a raw timestamp into useful features?	hour, day_of_week, is_weekend, is_holiday -> measure before/after impact
09 · Zomato	When does feature engineering cause data leakage?	Aggregates must use only data BEFORE the prediction timestamp
10 · Meta	200 features, 6hr training. How to reduce?	Drop near-zero-variance + correlation>0.95, then Lasso/tree importance
10 · Swiggy	RFE and Lasso disagree. Which to trust?	Match method to deployed model; validate with held-out accuracy
11 · General	KNN accuracy changes with/without scaling. Why?	Distance metric dominated by large-range features. Trees do not need scaling.
11 · Flipkart	MinMax vs Standard scaling -- when does MinMax fail?	Outliers compress MinMax range. StandardScaler more robust.
12 · General	Loss becomes nan. What first?	Lower learning rate by 10x -- diverging gradient steps overflow to inf/nan
12 · General	Explain what .fit() actually does.	Iterative gradient descent: predict -> measure loss -> gradient -> step downhill
13 · General	2 features 95% correlated -- coefficients?	Unstable, can flip sign. Predictions/R ² stay fine. Matters for interpretation only.
13 · General	RMSE >> MAE -- what does the gap mean?	A subset of predictions has unusually large errors -- check residuals
14 · General	95% accuracy fraud model -- check before celebrating?	Class distribution + confusion matrix + recall on minority class
14 · General	Why sigmoid, not clipped linear regression?	No flat zones, well-defined gradient everywhere, robust to outliers
15 · General	100% training accuracy -- good?	Red flag -- check train/test gap, constrain max_depth to 3-6
15 · General	Gini vs entropy -- does it matter?	Marginal difference; max_depth and feature engineering matter far more

SELF-TEST BEFORE WEEK 3

Without notes: explain the gradient descent loop in 4 steps. Compute Gini impurity for a group that is 70% class 1. Explain why a deployment scaler bug produces no error message. Name the 3 encoding strategies and when each is correct. If any feel shaky, go back to that day.

WEEK 2 IN ONE SENTENCE

"Every model -- from a single line to billions of parameters -- is the same loop: predict, measure, find the slope, step downhill. Once you see that loop, every algorithm is a variation on a theme."

Week 3 starts with Random Forests -> Gradient Boosting -> Model Evaluation

Full notebooks: github.com/VaishnaviJagtap18/42-Days-of-ML-Challenge · #42DaysOfML